CSC490 Assignment 5

Team Members
Minh Le (1007894432), Prashanth Shyamala (1008819021),
Shaan Purewal (1008332939), York Ng (1009153577)

1 Part One: Profiling Execution time

1.1 Objective

The objective of this work was to identify and optimize computational hotspots within the backend `process-results` pipeline. This pipeline transforms transcript data into embeddings and persists them to a database. Given that the system processes large transcripts and high-dimensional embeddings, even small inefficiencies can scale significantly.

We focused on five core functions along the transcript-to-embedding path and used profiling to distinguish true Python bottlenecks from external service latency. This ensured that optimization efforts targeted areas with meaningful performance impact rather than network-bound operations.

1.2 Methodology

Profiling was conducted using Python's `cProfile` with a dedicated harness to isolate each function under a consistent workload. Synthetic but realistic data was used to ensure that results reflected CPU and memory behavior rather than AWS latency.

External dependencies, including embedding generation and database execution, were replaced with local stubs. This design decision allowed us to:

- isolate Python-level computation costs,
- eliminate variability from network and service latency,
- produce reproducible and comparable measurements.

The workload simulated realistic production conditions, including thousands of transcript tokens, hundreds of segments, and high-dimensional embedding vectors.

1.3 Baseline Findings

Profiling results revealed that `insert_embeddings()` was the dominant computational hotspot. The primary cost originated from Python-level string construction when serializing embedding vectors.

Other functions exhibited smaller but measurable costs:

- `generate_text_embeddings()` incurred overhead from repeated UUID and timestamp generation,
- `insert_segments()` was dominated by deterministic UUID creation,
- transcript parsing functions showed linear scaling but relatively low cumulative cost.

These findings justified prioritizing optimization efforts on embedding serialization, where the expected payoff was highest.

1.4 Optimization Decisions

Optimizations were guided strictly by profiler evidence rather than speculative changes.

The most impactful change was replacing Python-level vector serialization:

```
"[" + ",".join(str(v) for v in embedding) + "]"
```

with:

```
json.dumps(embedding, separators=(",", ":"))
```

This decision was justified by two key factors:

- it eliminates hundreds of thousands of Python-level string conversions,
- it leverages a highly optimized standard library implementation.

Additional improvements included:

- hoisting loop-invariant operations to reduce repeated computation,
- reusing timestamps to avoid redundant formatting,
- simplifying numeric conversions and function lookups in tight loops.

Notably, an attempted refactor of `_combine_by_speaker()` was discarded after profiling showed performance regression. This reflects a deliberate decision to prioritize empirical evidence over perceived code improvements.

1.5 Results and Impact

The optimizations produced measurable improvements across multiple functions, with the most significant gain in embedding serialization:

- `insert_embeddings()`: $\sim 50\%$ reduction in execution time,
- `generate_text_embeddings()`: $\sim 13\%$ improvement,
- `_process_items()`: $\sim 15\%$ improvement,
- `insert_segments()`: modest improvement ($\sim 4\%$).

The impact is particularly important because these functions scale with transcript size, segment count, and embedding dimensionality. Reducing per-segment overhead directly lowers total processing time and improves system throughput.

Furthermore, the primary hotspot shifted from inefficient Python loops to optimized library internals, indicating that unnecessary overhead was successfully eliminated.

1.6 Limitations

Several limitations should be acknowledged:

- The profiling environment used mocked external services, so results do not reflect end-to-end latency.
- Network-bound costs, such as embedding API calls and database writes, were intentionally excluded.
- The workload, while realistic, may not capture all production edge cases such as extremely long transcripts or high concurrency.

As a result, while the optimizations improve CPU efficiency, overall system performance may still be constrained by external service limits.

1.7 Future Work

Future improvements should focus on architectural changes rather than further micro-optimizations:

- batching database writes to reduce per-segment overhead,
- reducing the number of embedding calls through caching or aggregation,
- exploring more efficient storage formats for high-dimensional vectors,
- re-evaluating chunking strategies to balance segment granularity and cost.

Additionally, profiling should be repeated under production-like conditions to validate that optimizations translate to real-world performance gains.

1.8 Conclusion

This work demonstrates the value of targeted profiling in performance optimization. By focusing on measured bottlenecks rather than assumptions, we achieved substantial improvements in critical execution paths while maintaining correctness.

The results highlight that in data-intensive pipelines, eliminating inefficient inner loops can yield disproportionately large gains, especially when operations scale multiplicatively with input size.

2 Part Two: Showing Code Coverage

To ensure that code coverage reporting was both reliable and visible, we automated coverage generation in GitHub Actions for both the backend and frontend components of the project. We adopted a CI-based approach rather than relying solely on local execution because locally generated results may vary across environments, can be overlooked during development, and are not immediately visible to reviewers. By integrating coverage reporting into the CI pipeline, coverage is computed consistently for each relevant change and surfaced directly in locations that support review and maintenance, namely pull requests and the repository `README`.

Backend Coverage

For the backend, we used **pytest-cov** and created a dedicated workflow in **.github/workflows/backend-coverage.yml**. This workflow performs the following tasks:

- installs the backend testing dependencies,
- executes the Python test suite on Python 3.13,
- generates coverage reports,
- uploads coverage artifacts,
- posts a persistent coverage comment on pull requests,
- updates the badge at `.github/badges/backend-coverage.svg`.

In addition, we added `pytest-cov` to `tests/requirements.txt` and introduced a `.coveragerc` configuration file. This configuration ensures that coverage is measured against `src/lambdas`, which corresponds to the actual backend implementation, rather than including the test suite itself. As a result, the reported coverage percentage more accurately reflects the extent to which the application code is exercised by automated tests.

Frontend Coverage

For the frontend, we used **Vitest**'s coverage tooling as an analogous solution. The workflow defined in **.github/workflows/frontend-coverage.yml** follows the same overall structure:

- install frontend dependencies,
- run Vitest with coverage enabled,
- upload coverage artifacts,
- report results in pull request comments,
- update the badge at `.github/badges/frontend-coverage.svg`.

Frontend coverage support was added through `frontend/package.json` and `frontend/vite.config.ts`. We intentionally aligned the frontend and backend workflows so that coverage reporting would remain consistent across the project. This improves maintainability and makes the reporting process easier for contributors and reviewers to follow.

How Results Are Reported

To ensure that the coverage results are both accessible and useful, they are reported in three forms:

- **workflow artifacts**, which preserve detailed coverage outputs for later inspection,
- **pull request comments**, which expose coverage results directly within the review process,
- **README badges**, which provide a concise summary of the current coverage status.

We also generalized `scripts/generate-coverage-badge.mjs` so that it can generate badges for both frontend and backend coverage summaries, despite the two tools producing different output formats. This reduced duplication in the reporting pipeline and improved its extensibility.

Impact

The primary impact of this work is that coverage reporting became automated, consistent, and continuously visible. Rather than existing as an isolated local metric, coverage was incorporated into the project's standard development and review workflow. This improves transparency, encourages testing discipline, and makes it easier to identify insufficiently tested changes before they are merged.

3 Part Three: Test Coverage Breadth

Rather than repeating the implementation details discussed in Part Two, this section presents the resulting coverage outcome and references the work through which it was achieved. Figure 2 shows the coverage reported by the automated workflows.

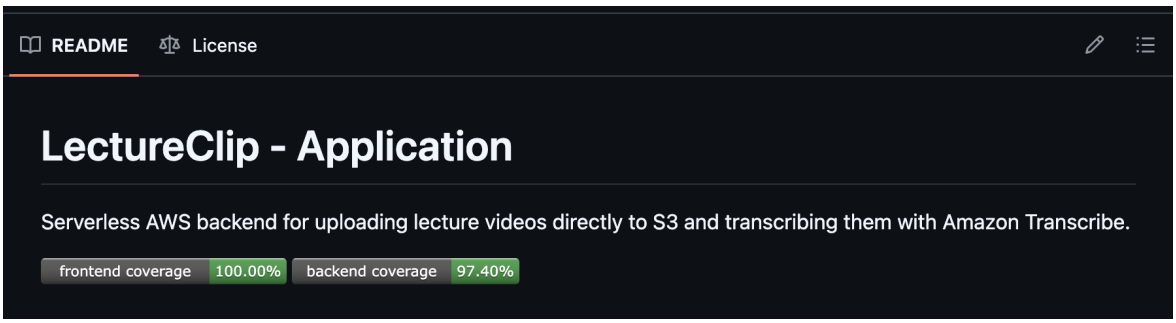

Figure 1: Coverage results reported as README badges (via Github workflow).

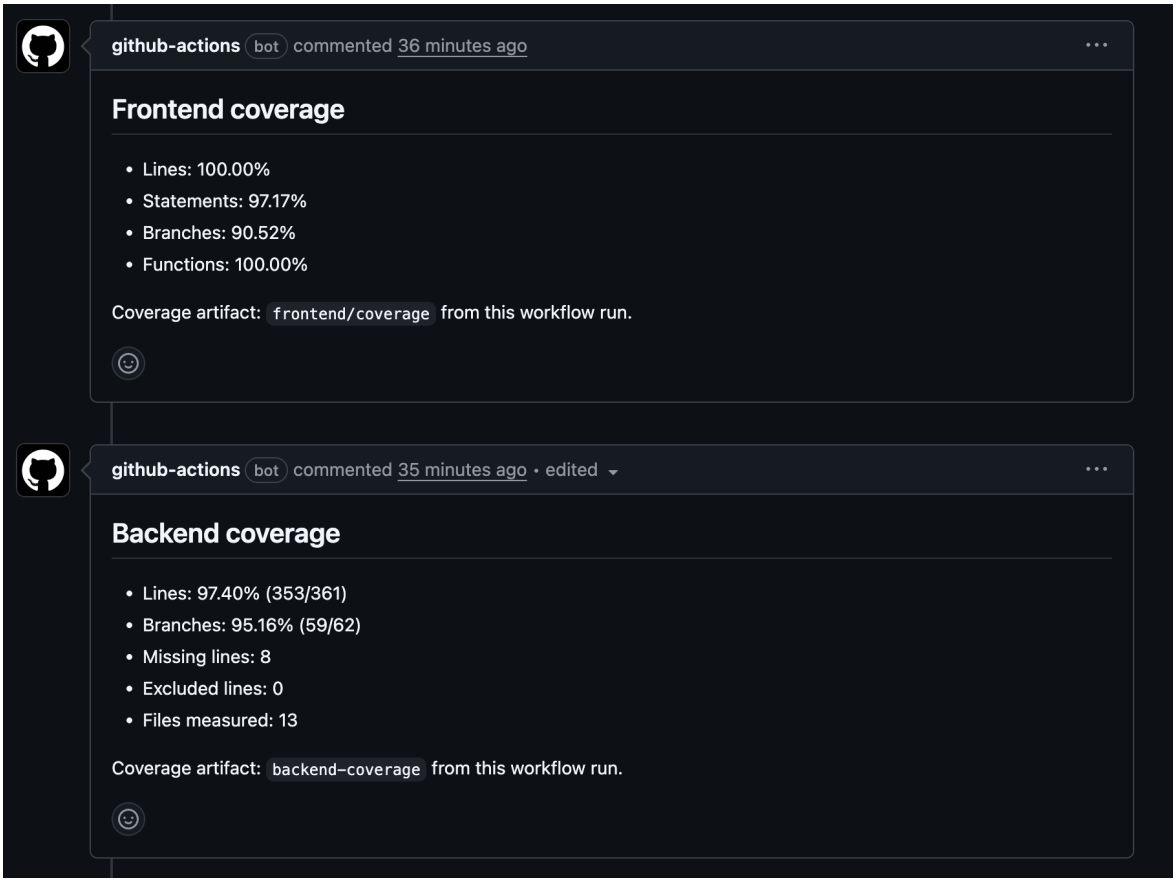

Figure 2: Coverage results reported as PR comments (via Github workflow).

The work associated with improving and reporting test coverage was carried out across two pull requests: the frontend coverage pull request (link) and the backend coverage pull request (link). Collectively, these changes introduced automated coverage generation, pull request reporting, repository badges, and the supporting configuration necessary to measure coverage in a meaningful and reproducible manner.

Instructions for running the coverage workflows locally and viewing the generated reports are provided in the root `README.md`. This ensures that the reported results are not only visible in CI, but can also be reproduced and independently verified.

4 Part Four: Test Coverage Breadth

The `process-results` module is the most complex module in the backend. It is the final stage of the transcription pipeline, invoked by Step Functions after process-transcribe signals success. It receives the URLs produced by Amazon Transcribe, converts the raw transcript into speaker-attributed text chunks, generates a semantic embedding vector for each chunk via Amazon Bedrock, then persists everything to Aurora PostgreSQL - making the lecture searchable by natural language.

`process-results` module link
Test file link

Test Suite — 30 Tests Across 6 Classes

The `process-results` module is validated through 30 tests spanning six test classes. These tests cover transcript parsing, chunking logic, embedding generation, database persistence, and full end-to-end handler execution. AWS services are mocked using `@mock_aws` (moto) where possible, while Bedrock is stubbed using a custom `_FakeBedrockRuntime` due to lack of moto support.

TestProcessItems (4 tests)

Tests the low-level transformation of raw Transcribe items into structured tuples.

- Verifies that pronunciation items are converted into $(\text{start_second}, \text{speaker}, \text{text})$ tuples
- Ensures punctuation tokens are appended to the preceding word rather than emitted separately
- Confirms no standalone punctuation tuples exist in the output
- Checks that speaker labels (e.g., `spk_0`, `spk_1`) are preserved

TestCombineBySpeaker (5 tests)

Validates chunking and grouping logic for speaker-attributed segments.

- Ensures output remains a list of 3-tuples
- Confirms consecutive tokens from the same speaker are merged
- Verifies that speaker changes trigger new chunks
- Tests handling of short trailing chunks (< 100 characters)
- Exercises large chunk splitting at sentence boundaries

TestFetchAndParseTranscript (4 tests)

End-to-end validation of transcript retrieval and parsing using mocked S3.

- Confirms the pipeline returns correctly structured tuples
- Validates types of tuple fields (integer timestamp, speaker string, non-empty text)
- Ensures correct S3 object is fetched even in the presence of decoy objects
- Verifies that missing objects raise appropriate exceptions

TestEmbedText (4 tests)

Tests the Bedrock embedding call using a stubbed runtime.

- Verifies correct `modelId` is passed to the API
- Checks that request body includes `inputText` and `dimensions`
- Confirms embedding vector is correctly extracted from the response

TestGenerateTextEmbeddings (7 tests)

Validates batch embedding generation and metadata construction.

- Ensures one embedding record per input segment
- Verifies presence of all required metadata fields
- Confirms embedding dimensionality
- Checks correct extraction of filename vs. full S3 URI
- Ensures text is preserved exactly
- Verifies UUID uniqueness across records

TestAuroraUtils (5 tests)

Tests database write logic using moto's RDS Data API mock.

- Validates UUID format and determinism for lecture IDs
- Ensures different URIs produce different lecture IDs
- Confirms segment insertion returns correct number of records
- Verifies segment timing logic (next start time or +30 second heuristic)
- Checks deterministic segment ID generation
- Includes a smoke test ensuring embedding insertion completes without error

TestHandler (6 tests)

Full integration tests combining S3, parsing, embedding, and database layers.

- Verifies successful execution returns HTTP 200 response
- Confirms at least one segment is produced from valid transcripts
- Ensures embedding count matches segment count (1:1 mapping)
- Validates failure when `transcriptUrl` is missing
- Confirms compatibility with both `mediaUrl` and `s3_uri` event formats

5 Part Five: Two Memorable Bugs

Video Walkthrough

S3 Object Key Encoding Bug One issue encountered during development involved how Amazon S3 encodes object keys in event notifications.

When a file is uploaded to S3 and triggers a Lambda, the object key is URL-encoded rather than passed in its raw form. For example, a file named `"lecture video.mp4"` may appear in the event as `"lecture%20video.mp4"` or `"lecture+video.mp4"`, depending on the encoding.

The bug arose because this encoded key was passed directly into the Step Functions execution input when constructing the S3 URI. Downstream, the `process-results` module attempted to look up the corresponding record in Aurora using this URI. However, the URI stored at upload time used the decoded (human-readable) version with spaces, leading to mismatches and failed lookups.

The fix was to decode the key immediately after extracting it from the S3 event. In the `s3-trigger` Lambda, we applied:

```
urllib.parse.unquote_plus(key)
```

To prevent regressions, a dedicated test, `test_url_encoded_key_is_decoded`, was added. The test constructs an S3 event with a deliberately encoded key:

```
"user%20123/lecture+video.mp4"
```

After invoking the handler, the Step Functions execution input is inspected to verify that the decoded values appear:

- `"user 123"`
- `"lecture video.mp4"`

If the decoding step is removed, this test fails immediately, ensuring the bug cannot reoccur unnoticed.

Terraform State Corruption Bug A second issue arose during the migration from a single-environment setup to a dual-environment infrastructure configuration managed by Terraform.

Terraform relies on a state file (stored in S3) to track infrastructure resources. During the transition, the existing state file was removed from the codebase. When the new infrastructure was deployed, Terraform initialized a new state that did not match the existing resources in AWS. This caused the remote state in S3 to become inconsistent, effectively corrupting the entire infrastructure stack.

Because the state was no longer aligned with actual resources, the issue could not be resolved through the Terraform CLI. The only viable solution was to manually delete the affected resources using the AWS Management Console and then perform a clean redeployment.

This incident highlighted an important operational guideline: when a change attempts to modify or delete the Terraform state file, the safest recovery strategy is often to destroy the existing stack and redeploy from a clean state.